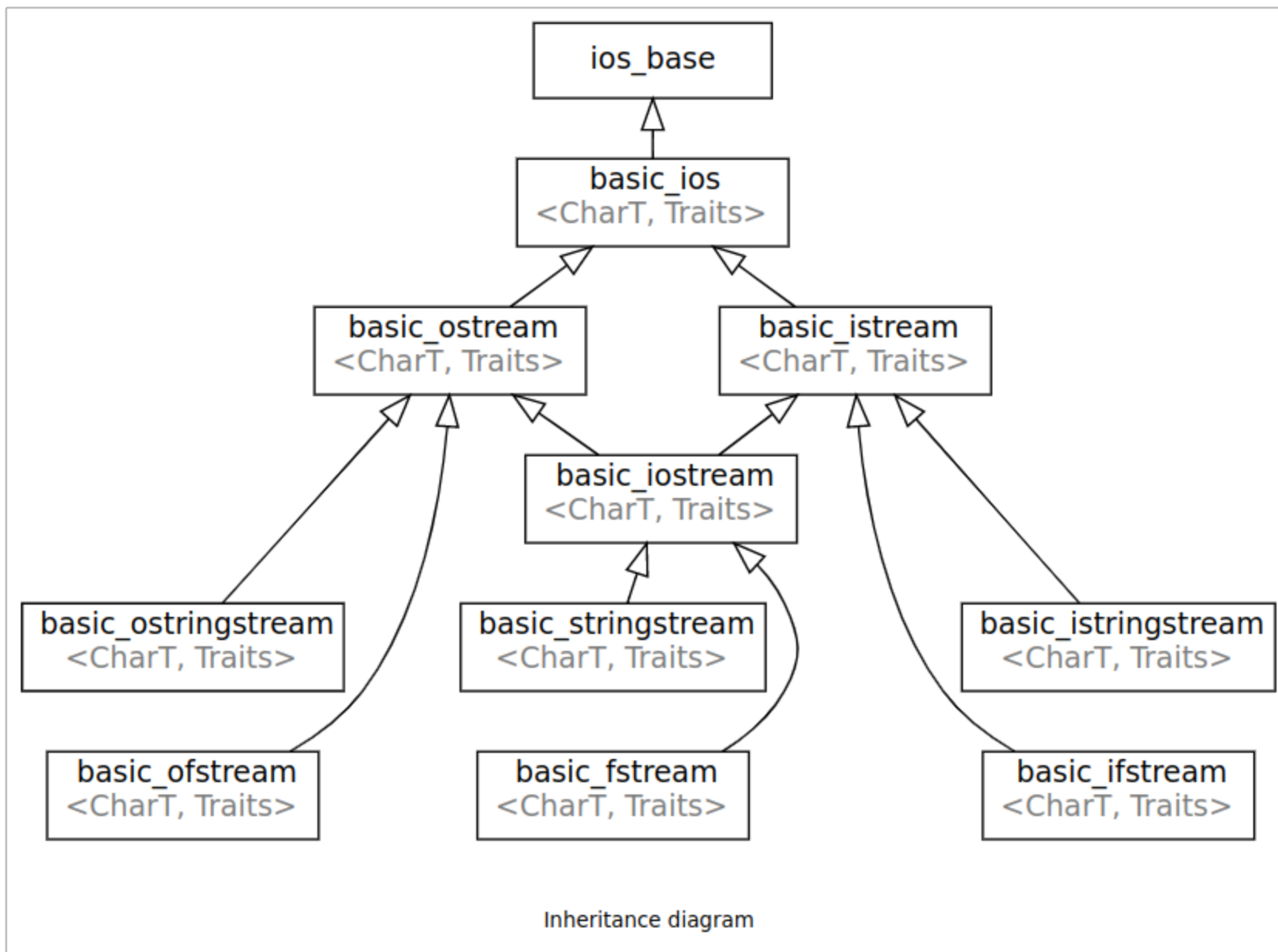


GKxx

Contents

- IOStream
- STL
 - 容器
 - 迭代器
 - 迭代器适配器

注：以下所有来自标准库的名字可能会省去 `std::`，但不代表我在代码里也会省去。



->

IOStream

`ifstream`, `ofstream`: 文件输入流, 文件输出流, 定义在 `<fstream>` 里

一般的文件流 (默认情况下既可输入又可输出): `fstream`

从文件读入:

```
std::ifstream ifs("path/to/myfile.ext");  
int ival; std::string s;  
ifs >> ival >> s;
```

向文件写入:

```
std::ofstream ofs("path/to/myfile.ext");  
ofs << "hello world" << std::endl;
```

fstream

打开文件的模式: `app`, `binary`, `in`, `out`, `trunc`, `ate`, (since C++23) `noreplace`

https://en.cppreference.com/w/cpp/io/basic_ifstream#Member_types_and_constants ->

通过 `|` 可以将多个模式并起来, 例如 `std::ios::out | std::ios::binary`。

默认情况下: `ifstream` 使用 `in`, `ofstream` 使用 `out`, `fstream` 使用 `in | out`。

- 假如你传入的打开模式是 `mode`, `ifstream` 会以 `mode | in` 打开, `ofstream` 会以 `mode | out` 打开。
- `out` 会清空这个文件原来的内容, 而 `app` 是将输出追加到原来的内容的后面。

```
std::ofstream file("myfile.txt", std::ios::app);  
file << "hello world" << std::endl;
```

fstream

可以在构造的时候打开文件，并同时指定打开模式；也可以在稍后用 `open` 打开一个文件，或用 `close` 关闭打开的文件：

```
std::ifstream file("myfile.txt", std::ios::binary);
if (!file.is_open()) {
    // myfile.txt is not found or cannot be opened.
    // handle this error.
    file.open("myfile.txt", std::ios::binary);
}
// do something with the file
file.close();
file.open("another_file.txt");
// ...
```

`file` 的析构函数会关闭打开的文件。

stringstream

将一个 `std::string` 作为输入内容或输出结果

`stringstream`, `ostringstream`, `stringstream`: 定义在 `<sstream>` 里。

```
std::ostringstream oss;  
oss << 42 << 3.14 << "hello world";  
std::string str = oss.str(); // "423.14hello world"
```

注：以下所有来自标准库的名字可能会省去 `std::`，但不代表我在代码里也会省去。

STL 概览

诞生于 1994

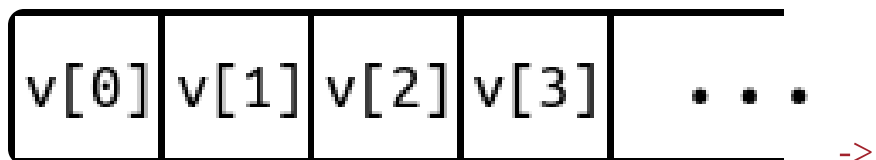
- 容器 containers: 顺序 (sequence) 容器, 关联 (associative) 容器
- 迭代器 iterators
- 算法 algorithms
- 适配器 adaptors: iterator adaptors, container adaptors
- 仿函数 function objects (functors)
- 空间分配器 allocators

容器

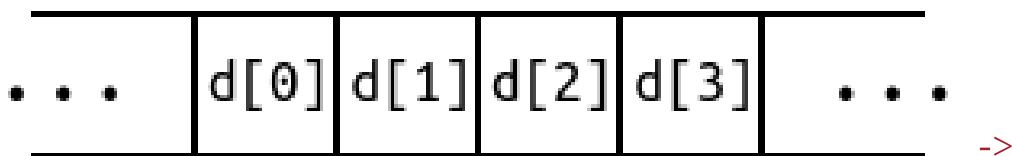
- 顺序容器 sequence containers
 - vector, deque, list, array, forward_list
- 关联容器 associative containers
 - map, set, multimap, multiset
 - unordered_map, unordered_set, unordered_multimap, unordered_multiset

顺序容器

- `vector<T>`: 可变长数组, 支持在末尾快速地添加、删除元素, 支持随机访问 (random access)

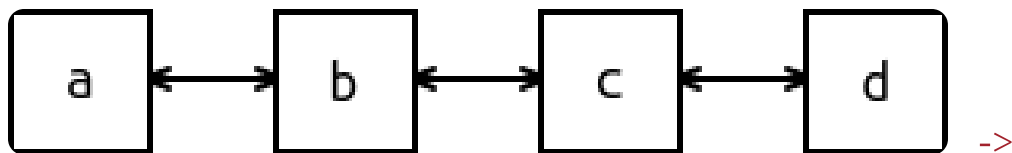


- `deque<T>`: 双端队列 (**d**ouble **e**nded **q**ueue), 支持在开头和末尾快速地添加、删除元素, 支持随机访问

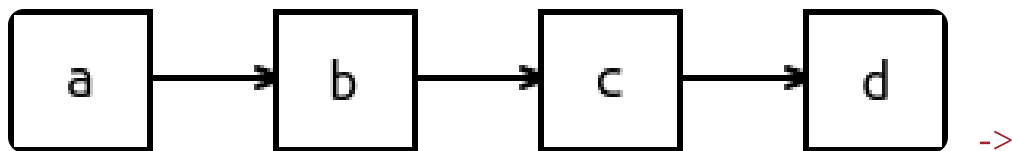


顺序容器

- `list<T>`: 双向链表, 支持在任意位置快速地添加、删除元素, 支持双向遍历, 不支持随机访问



- `forward_list<T>`: 单向链表, 支持在任意位置快速地添加、删除元素, 仅支持单向遍历, 不支持随机访问



- `array<T, N>`: 内置数组 `T[N]` 的套壳, 提供和其它 STL 容器一致的接口 (包括迭代器等), 可以直接拷贝, 不会退化为 `T*`。
- 特别的: `string` 不是容器, 但非常像一个容器

统一的接口：构造

1. `Container c`: 一个空的容器
2. `Container c(n, x)`: n 个 x
3. `Container c(n)`: n 个元素, 每个元素都被值初始化
 - `string` 不支持这一操作, 为什么?
4. `Container c(begin, end)`: 从迭代器范围 `[begin, end)` 中拷贝元素。

`array<T, N>` 只支持 1, 为什么?

统一的接口：完整列表 ->

相同的功能在所有容器上都具有相同的接口（除了 `forward_list` 有一点点特殊）

- 拷贝构造、拷贝赋值、移动构造、移动赋值
- `c.size()`, `c.empty()`, `c.resize(n)`
- `c.capacity()`, `c.reserve(n)`, `c.shrink_to_fit()`
- `c.push_back(x)`, `c.emplace_back(args...)`, `c.pop_back()`
- `c.push_front(x)`, `c.emplace_front(args...)`, `c.pop_front()`
- `c.at(i)`, `c[i]`, `c.front()`, `c.back()`
- `c.insert(pos, ...)`, `c.emplace(pos, args...)`, `c.erase(...)`, `c.clear()`

统一的接口

记忆的关键：

1. 理解每一种容器的底层数据结构，自然就明白为何支持/不支持某种操作
 - 为何链表不支持下标访问？为何 `vector` 不支持在开头添加元素？
2. 记住这些接口的名字
3. 如果一个操作应该被支持，它就必然叫那个名字、是那个用法

string 和 vector 的“容量”

string 大概率 和 vector 采用类似的增长方式，分配的内存可能比当前存储的元素所占用的内存大。

- 当前所拥有的内存能放下多少个元素，称为“容量” (capacity)，可以通过 `c.capacity()` 查询。
- `c.reserve(n)` 为至少 `n` 个元素预留内存。
 - 如果 `n <= c.capacity()`，什么都不会发生。
 - 否则，重新分配内存使得 `c` 能装得下至少 `n` 个元素。
 - **务必区分 reserve 和 resize。**
- `c.shrink_to_fit()`：**请求** `c` 释放多余的容量（可能重新分配一块更小的内存）
 - 这个函数在 `deque` 上也有。

insert 和 erase

`c.insert(pos, ...)`, `c.emplace(pos, args...)` , 其中 `pos` 是一个迭代器。

在 `pos` 所指的位置之前添加元素。

- `insert` 有很多种写法, 可以 `c.insert(pos, x)`, `c.insert(pos, begin, end)`, `c.insert(pos, {a, b, c, ...})` 等等, 用到的时候再查。

`c.erase(...)` 也有很多种写法, 用到的时候再查。

特殊的 `forward_list`

`forward_list` 的功能完全被 `list` 包含，那为何我们还需要 `forward_list`？

特殊的 `forward_list`

`forward_list` 的功能完全被 `list` 包含，那为何我们还需要 `forward_list`？

为了省时间，省空间。

- 单向链表的结点比双向链表的结点少存一个指针
- 维护单向链表上的链接关系也比维护双向链表少一些操作

因此，`forward_list` 采取最简的实现：能省则省

- 它不能 `push_back/pop_back`。
 - 如果需要，你可以自己保存指向末尾元素的迭代器，然后用 `insert_after`
- 它甚至不支持 `size()`。如果需要，你可以自己用一个变量记录。

特殊的 `forward_list`

`insert`, `emplace` 和 `erase` 变成了 `insert_after`, `emplace_after` 和 `erase_after`

- 单向链表上，操作“下一个元素”比操作“当前元素”或“前一个元素”更方便。

越界检查

`c.at(i)` 在越界时抛出 `std::out_of_range` 异常

`c[i]`, `c.front()`, `c.back()`, `c.pop_back()`, `c.pop_front()` 统统不检查越界，一切为了效率。

但是我们在自己设计自己的容器时，不一定要采用标准库的这种方式。

- 也许下面这种设计更合理？

```
auto &operator[](size_type n) {  
    assert(n < size());  
    return data[n];  
}
```

- `assert` 在 Debug 模式下生效，而在 Release 模式下（定义了 `NDEBUG` 宏）无效，不会影响 Release 模式下的效率。

选择正确的容器

顺序容器：

- 能维持元素的先后顺序
- 某些情况下的插入、删除、查找可能较慢

关联容器（不带 `unordered` 的）：

- 元素总是有序的，默认是升序（因为底层数据结构通常是红黑树）
- 支持 $O(\log n)$ 地插入、删除、查找元素

无序关联容器（`unordered`）：

- 元素是无序的（因为底层数据结构是哈希表）
- 支持平均情况下 $O(1)$ 地插入、删除、查找元素

复习：假如没有迭代器...

不同的容器，根据底层数据结构不同，遍历方式自然也不同：

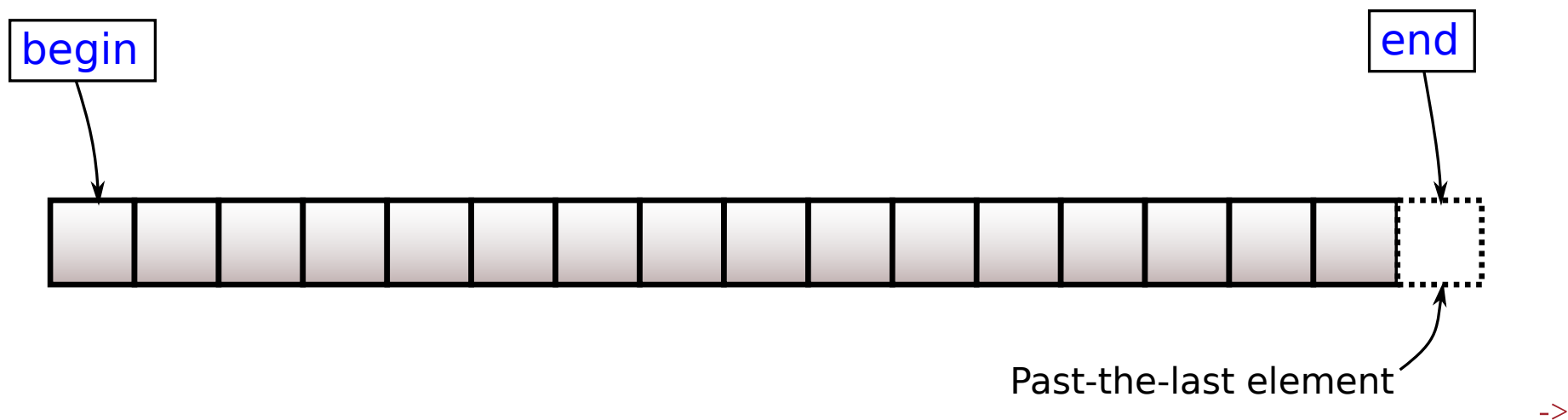
```
for (std::size_t i = 0; i != a.size(); ++i)
    do_something(a[i]);
// 可能的方式：通过指向结点的“句柄”（指针）遍历一个链表
for (node_handle node = l.first_node(); node; node = node.next())
    do_something(node.value())
```

如果是个更复杂的容器呢，比如基于哈希表/红黑树实现的关联容器？

复习：使用统一的方式访问元素、遍历容器

所有容器都有其对应的迭代器类型 `Container::iterator`，例如 `std::string::iterator`，`std::vector<int>::iterator`。

所有容器都支持 `c.begin()`，`c.end()`，分别返回指向**首元素**和指向**尾后位置**的迭代器。



* 我们总是使用左闭右开区间 `[begin, end)` 表示一个“迭代器范围”

复习：使用统一的方式访问元素、遍历容器

- `it1 != it2` 比较两个迭代器是否相等（指向相同位置）
- `++it` 让 `it` 指向下一个位置。
- `*it` 获取 `it` 指向的元素的引用。

```
for (auto it = c.begin(); it != c.end(); ++it)
    do_something(*it);
```

基于范围的 `for` 语句：完全等价于上面的使用迭代器的遍历

```
for (auto &x : c)
    do_something(x);
```

const_iterator

带有“**底层 const**”的迭代器。

- `Container::const_iterator`
- `c.cbegin()`, `c.cend()`
- 在一个 `const` 对象上, `c.begin()` 和 `c.end()` **也返回** `const_iterator`。

对 `const_iterator` 解引用会得到 `const T &` 而非 `T &`, 无法通过它修改元素的值。

begin, end, cbegin, cend

再次出现了 `const` 和 `non-const` 的重载。

```
class MyContainer {  
public:  
    using iterator = /* unspecified */;  
    using const_iterator = /* unspecified */;  
    iterator begin();  
    const_iterator begin() const;  
    iterator end();  
    const_iterator end() const;  
    const_iterator cbegin() const;  
    const_iterator cend() const;  
};
```

复习：迭代器型别 (iterator category)

- ForwardIterator: 支持 `*it`, `it->mem`, `++it`, `it++`, `it1 == it2`, `it1 != it2`
- BidirectionalIterator: 在 ForwardIterator 的基础上, 支持 `--it`, `it--`。
- RandomAccessIterator: 在 BidirectionalIterator 的基础上, 支持 `it[n]`, `it + n`, `it += n`, `n + it`, `it - n`, `it -= n`, `it1 - it2`, `<`, `<=`, `>`, `>=`。

`vector`, `string`, `array`, `deque` 的迭代器是 `RandomAccessIterator`; `list` 的迭代器是 `BidirectionalIterator`; `forward_list` 的迭代器是 `ForwardIterator`。

复习：迭代器型别 (iterator category)

还有两种迭代器型别：InputIterator 和 OutputIterator。

- InputIterator 表示**可以通过这个迭代器获得元素**（不要求能修改它所指向的元素）
- OutputIterator 表示**可以通过这个迭代器写入元素**（不要求能获得它所指向的元素）
- A ForwardIterator **is an** InputIterator.

稍后我们会看到一些例子。

迭代器的辅助函数

- `std::advance(iter, n)` 将 `iter` 前进 `n` 步。 `n` 也可以是负的。
- `std::distance(iter1, iter2)` 返回 `iter` 和 `iter2` 之间的**距离**：相距几个元素。
- `std::next(iter)` 返回 `iter` 的“下一个位置”， `std::prev(iter)` 返回 `iter` 的“上一个位置”，它们都不会改变 `iter` 本身。
 - 用来代替手动的 `auto tmp = iter; ++tmp;`。

以上函数对于**各类迭代器型别**都有支持，并且会针对不同的型别提供不同的实现。

- 不管 `iter1` 和 `iter2` 是什么型别，你都可以用 `std::distance(iter1, iter2)` 获取它们的距离，不用担心 `iter1 - iter2` 这个表达式能不能编译之类的问题。

迭代器适配器 (iterator adaptors)

一种用起来像迭代器的东西

- 可能是根据某个迭代器和/或一些别的东西“变出来”的
- 用起来像迭代器，但多多少少有点区别，比如
 - 反向迭代： `++` 实际上是 `--`
 - 帮助移动元素： `*iter` 返回右值引用
 - 看似迭代，实则插入元素： `*iter++ = x` 会被“变”成 `c.push_back(x)`
 - 将某些特殊的过程也抽象为“迭代”： `x = *iter++` 被“变”成 `std::cin >> x`
 -

反向迭代器 `reverse_iterator`

每个容器都有自己的反向迭代器: `Container::reverse_iterator` 和 `Container::const_reverse_iterator` , 通常被这样定义:

```
using reverse_iterator = std::reverse_iterator<iterator>;  
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
```

- `c.rbegin()`, `c.rend()`, `c.crbegin()`, `c.crend()`
- `++` 和 `--` 在反向迭代器上都是反的。

```
std::vector v{1, 2, 3, 4, 5};  
for (auto rit = v.rbegin(); rit != v.rend(); ++rit)  
    std::cout << *rit << ' ';
```

输出: 5 4 3 2 1

移动迭代器 `move_iterator`

- `std::make_move_iterator(iter)` 从一个普通的迭代器 `iter` 变出一个“移动迭代器”
- `*mit` 会得到右值引用而非左值引用，从而元素更可能被移动而非被拷贝。

```
std::vector<std::string> words = someValues();  
std::vector<std::string> v(words.size());  
std::copy(std::make_move_iterator(words.begin()),  
          std::make_move_iterator(words.end()), v.begin());
```

`words` 中的每个 `string` 被**移动**进了 `v`，而不是拷贝。

从迭代器向迭代器的映射

对于一个给定的迭代器类型 `Iter` , `std::reverse_iterator<Iter>` 和 `std::move_iterator<Iter>` 都是一个新的迭代器类型:

- 它们是 `Iter` 的“wrapper”: 在内部保存一个 `Iter` 类型的原始迭代器, 可以通过 `rit.base()` 访问。
- 它们基于 `Iter` 的接口定义自己的接口: `++rit` 会调用 `--it` , `*mit` 会返回 `std::move(*it)`

C++23 还有个 `std::basic_const_iterator<Iter>` , 用类似的方式定义了 `Iter` 的带有底层 `const` 的版本。

你当然可以用类似的方式实现自己的特殊迭代器。

插入迭代器

`insert_iterator`, `front_insert_iterator`, `back_insert_iterator`

典型的 OutputIterator:

- 只可向 `*iter` 写入元素, 不能从 `*iter` 读取元素
- 它们会调用容器的 `insert`, `push_front` 或 `push_back`, 将“写入”的元素插入容器

```
std::vector<int> numbers = someValues();  
std::vector<int> v;  
std::copy(numbers.begin(), numbers.end(), v.begin()); // 错误!  
std::copy(numbers.begin(), numbers.end(), std::back_inserter(v)); // 正确
```

`std::back_inserter(c)` 生成一个 `std::back_insert_iterator<Container>`, 它内部保存一个 `Container &r = c`, 会不断调用 `r.push_back(x)` 将向它“写入”的元素添加进 `c`。

流迭代器

读入一串数存进一个 `vector<int>`:

```
std::vector<int> v(std::istream_iterator<int>(std::cin),  
                  std::istream_iterator<int>{});
```

利用 `std::copy` 将 `v` 中的元素输出，并且每个后面跟一个 `", "`:

```
std::copy(v.begin(), v.end(), std::ostream_iterator<int>(std::cout, ", "));
```

- `istream_iterator` 是一种 `InputIterator`，它不断从输入流中获取元素
- `ostream_iterator` 是一种 `OutputIterator`，它不断将向它“写入”的元素写进输出流

迭代器型别

InputIterator 和 ForwardIterator 都要求支持 `++it`, `it++`, `*it`, `it->mem`, `==`, `!=`。

这两类迭代器的区别究竟是什么？

迭代器型别

InputIterator 和 ForwardIterator 都要求支持 `++it`, `it++`, `*it`, `it->mem`, `==`, `!=`。

这两类迭代器的区别究竟是什么？—— ForwardIterator 提供 **multi-pass guarantee**

```
auto original = iter;           // 对当前的 iter 做个拷贝
auto value = *iter;             // 现在 iter 指向的元素是 value
++iter;
assert(*original == value);
```

对于一个 ForwardIterator 来说，`original` 指向了 `iter` 在递增之前指向的位置，那个位置上的值始终是 `value`。

InputIterator 不这么认为，它只保证能“input”：`++iter` 就意味着我们打算获取“下一个值”了，先前的值也就无法再被获取了。（考虑“输入”的过程）